TECHNICAL REPORT • 2026

基于 NapCat 与 NoneBot 的 QQ 群聊智能体系统

面向群聊自治参与、长上下文记忆与表情包学习的
架构设计、工程实现与云上部署

项目名称	qq-ai-groupmate
报告版本	v1.0
报告日期	2026 年 10 月 28 日
作者	miaom4702-art
代码仓库	github.com/miaom4702-art/qq-ai-groupmate
许可协议	MIT（第三方组件遵循各自许可）

本报告中的服务器地址、QQ 账号、群号、API Key 与 token 均已脱敏。QQ 为腾讯专有产品，使用第三方协议端存在账号风险，请使用小号并遵守相关条款。

文档信息与摘要

文档类型	系统技术报告	**适用读者**	研发、运维、项目评审
系统形态	QQ 群聊智能体（NapCat + OneBot v11 + NoneBot2）	**部署形态**	Windows 本地 / 阿里云 ECS Docker
核心模型	Qwen3.7-Plus / Qwen-Flash / Qwen3-VL-Plus / text-embedding-v4	**数据存储**	SQLite + Qdrant + 本地文件

摘要

本项目实现了一个能够在 QQ 群聊中自主观察、判断并参与对话的智能体。系统采用 NapCatQQ 作为 QQ 协议端， 通过 OneBot v11 WebSocket 接入 NoneBot2，并以成熟开源插件 nonebot-plugin-ai-groupmate 作为群聊智能体核心。 在工程上，系统将“是否接话”的决策与“如何回复”的生成解耦：先由低成本的 Gatekeeper 判断语境， 再由主模型结合最近消息、群体认知档案、明确记忆与 RAG 检索生成回复；表情包则经过收集、去重、视觉标注、 文本向量化与语义检索后，在合适语境中发送。项目还针对单群活跃度、Bot 身份识别、消息防抖、连续对话窗口、 表情包收录阈值与人设提示词做了可复现的小幅优化，并完成了从 Windows 本地到阿里云 Ubuntu + Docker Compose 的迁移部署。测试结果表明，系统可稳定完成 QQ 登录、消息收发、群聊自主回复、表情包发送、数据库自动迁移 与 Web 管理面板访问；在 1.6 GiB 内存的 ECS 上，Bot、NapCat、Qdrant 三个容器合计占用约 595 MiB， 未触发 swap。公网 18080 端口仍待安全组放行，是当前唯一的部署待办项。

关键词：QQ 群聊智能体；NapCat；OneBot v11；NoneBot2；Gatekeeper；RAG；上下文压缩；表情包学习；Docker

目录

1. 项目概述
2. 需求分析
3. 总体架构
4. 关键技术与设计
5. 面向群聊体验的增量优化
6. 部署与运维
7. 测试与结果
8. 安全、合规与风险
9. 局限性与后续工作
10. 结论
11. 参考文献
12. 附录

1. 项目概述

1.1 背景与动机

传统 QQ 机器人通常依赖关键词、命令或显式 @ 触发，交互方式偏“工具化”。在真实群聊中，用户更希望 Bot 像一个普通群友：能看懂当前话题，判断自己是否适合插话，记得群里发生过什么，并在合适的时候使用表情包。这类需求同时涉及协议接入、事件处理、对话决策、长期记忆、检索增强生成（RAG）、多模态理解与部署运维， 单点实现容易，系统化稳定运行较难。

项目不重复造轮子，而是组合成熟开源方案：NapCatQQ 解决 QQ 协议接入，NoneBot2 解决事件与插件生态，nonebot-plugin-ai-groupmate 提供群聊特化的 Agent、Gatekeeper、群体记忆、上下文压缩、RAG 与表情包学习， Qdrant 提供本地向量检索，阿里云百炼提供统一的模型能力。项目工作集中在“可复现的配置、少量关键补丁、 安全部署与验证”上。

1.2 建设目标

- **自治参与**：无需 @，按群聊语境与概率自主判断是否发言，并受冷却与频率限制约束。
- **话题理解**：结合最近消息、群体档案、明确记忆与历史 RAG，理解当前话题而不是只看最后一句话。
- **上下文压缩**：长群聊不无限堆叠原文，使用摘要、记忆与检索控制 token 预算。
- **表情包学习**：自动收集群内常用表情包，使用视觉模型标注情绪与使用场景，在合适语境中检索发送。
- **可运维**：本地可开发、云端可部署，具备日志、WebUI、健康检查与自动数据库迁移。
- **安全与合规**：密钥不入库、端口最小暴露、危险工具默认关闭、使用小号并提示平台风险。

1.3 主要工作

- 完成 NapCat + OneBot v11 WebSocket + NoneBot2 的端到端链路搭建与 QQ 扫码登录。
- 接入 ai-groupmate 2.5.16，配置 Gatekeeper、群体记忆、RAG、表情包学习与百炼模型角色。
- 实现单群活跃度补丁：允许重点群突破上游 0.1 的主动抽样上限，同时不影响其他群。
- 完成身份、昵称、人设、防抖、连续对话与表情包阈值的配置级优化。
- 将系统容器化并部署到阿里云 Ubuntu 24.04 ECS，新增 18080 作为 Bot WebUI 端口。
- 完成功能、资源、安全与故障恢复测试，并整理为可复现的开源仓库。

1.4 术语

术语	说明
OneBot v11	QQ 机器人协议标准，定义事件、消息段与 API。
NapCatQQ	基于 NTQQ 的协议端实现，向 NoneBot 提供 OneBot v11 WebSocket。
Gatekeeper	低成本判断模型，用于决定某条群消息是否值得进入主 Agent 回复。
RAG	检索增强生成，将历史聊天/知识检索结果注入模型上下文。
VLM	视觉语言模型，用于表情包内容理解与标注。
Qdrant	向量数据库，存储聊天与表情包的语义向量。
群体认知档案	由模型维护的群长期摘要，记录话题、成员特征、内部梗与氛围。

2. 需求分析

2.1 功能需求

编号	需求	验收要点
F1	群消息接入	NapCat 登录后，NoneBot 能稳定接收群文本、图片、回复、@、转发等事件。
F2	自主接话	普通消息按群级概率进入 Gatekeeper；重点群可单独提高概率；不依赖 @。
F3	话题理解	回复能结合最近上下文、群档案、明确记忆与历史检索，而不是孤立回复。
F4	上下文压缩	长群聊下 token 可控；摘要与 RAG 可按需更新，不阻塞消息处理。
F5	表情包	可收集、去重、视觉标注、向量化，并在合适语境中发送。
F6	人设与风格	人设可配置；Bot 名称/昵称与群友称呼一致；避免客服腔与长篇说教。
F7	管理面板	可查看 token 用量、运行状态与配置；有访问 token 保护。
F8	部署与迁移	支持 Windows 本地运行与 Linux Docker 部署；数据卷持久化。

2.2 非功能需求

维度	要求	实现方式
稳定性	断线重连、容器自恢复、数据库迁移幂等	NoneBot 重连；Docker restart；entrypoint 启动前执行 ORM upgrade
性能	低配 ECS 可运行；消息处理不阻塞	Gatekeeper 小模型；异步向量化；Qdrant 本地部署
安全	密钥不泄漏；端口最小暴露	.env 忽略；仅 18080 计划公网；3001/6333 走容器网络
成本	单群每日调用可控	群日限额、冷却、频率上限、低概率全局抽样
可维护	配置化、可复现、可开源	补丁文件、bootstrap 脚本、Dockerfile、README、NOTICE

2.3 约束与假设

- QQ 协议端非官方，存在风控与账号限制风险；必须使用小号。
- 模型 API 需可访问；本项目使用阿里云百炼 OpenAI 兼容接口。
- NapCat 对 QQ 版本敏感；本项目使用 OneKey 便携版 QQ 9.9.33 + NapCat 4.18.41。
- 服务器资源有限：1.6 GiB RAM + 2 GiB swap，需要控制镜像与容器数量。

3. 总体架构

3.1 分层架构

接入层 QQ 客户端（Bot 小号） → NapCatQQ（OneBot v11）

↓ **WebSocket（正向 / 反向）**

框架层 NoneBot2 2.5 + nonebot-adapter-onebot 2.4.6（事件分发、Matcher、API 调用）

↓

智能体层 nonebot-plugin-ai-groupmate 2.5.16：Gatekeeper、Agent、工具调用、群体记忆、RAG、表情包

↓

模型层 阿里云百炼：Qwen3.7-Plus / Qwen-Flash / Qwen3-VL-Plus / text-embedding-v4

↓

数据层 SQLite（消息/记忆/配置） + Qdrant（聊天与表情包向量） + 本地文件（图片/密钥）

↓

运维层 Docker Compose、日志、WebUI、健康检查、安全组

3.2 组件职责

组件	职责	关键配置
NapCatQQ	登录 QQ、收发消息、提供 OneBot v11 API/事件	WebSocket 3001、WebUI 6099、token
NoneBot2	事件循环、插件加载、Matcher 分发、HTTP/WS 驱动	`~fastapi + ~httpx + ~websockets`
ai-groupmate	群聊决策、Agent 工具、记忆、RAG、表情包、定时任务	模型角色、概率、限额、人设
Qdrant	聊天历史与表情包描述的向量检索	集合 `chat_collection_v2`、`media_collection_text`
SQLite	消息、明确记忆、群档案、配置、定时任务	`data/ai_groupmate.db`
百炼	对话、判断、摘要、视觉标注、Embedding	OpenAI 兼容接口

3.3 消息处理时序

1. 接收	2. 接入	3. 抽样	4. 判断
QQ 消息进入 NapCat，转为 OneBot 事件	NoneBot 解析事件、去重、消息防抖	按群级概率决定是否进入判断	Gatekeeper 输出 should_reply / recipient
5. 组装	**6. 生成**	**7. 发送**	**8. 异步**
最近消息 + 群档案 + 记忆 + RAG	主模型决定回复/工具调用/表情包	OneBot API → NapCat → QQ	向量化、摘要、档案更新、表情标注

第 3 步的抽样是“是否值得让 Gatekeeper 看一眼”，不是“直接回复”。第 4 步仍会结合语境拒绝不合适的插话； 第 6 步才真正消耗主模型。该分层是本系统控制成本与减少“没话找话”的关键。

3.4 部署拓扑

位置	组件	端口 / 暴露
公网	Bot / 插件 WebUI	`18080 → 8080`，待安全组放行
服务器本机	NapCat WebUI	`127.0.0.1:16099 → 6099`，SSH 隧道访问
Docker 网络	OneBot WebSocket	`napcat:3001`，不暴露公网
Docker 网络	Qdrant	`qdrant:6333`，不暴露公网

4. 关键技术与设计

4.1 OneBot v11 与 WebSocket

NapCat 将 QQ 事件转换为 OneBot v11 标准事件，通过 WebSocket 推送给 NoneBot。项目中 NoneBot 作为客户端， 主动连接 NapCat 的 WebSocket 服务端；两者通过 `ONEBOT_ACCESS_TOKEN` 共享 token。消息段采用 `array` 格式，便于处理文本、图片、@、回复、转发等复合消息。

```
# bot/.env（节选，值已脱敏）
DRIVER=~fastapi+~httpx+~websockets
ONEBOT_WS_URLS=["ws://napcat:3001"]
ONEBOT_ACCESS_TOKEN=<RANDOM_TOKEN>
```

选择 WebSocket 而不是 HTTP 轮询，是为了获得低延迟事件推送；在 Docker 部署中，Bot 与 NapCat 位于同一 Compose 网络，3001 不映射到公网，减少攻击面。

4.2 NoneBot 插件与 Agent

NoneBot2 负责事件循环与 Matcher 分发，ai-groupmate 在 Matcher 中完成消息入库、抽样、Gatekeeper、Agent 执行与发送。Agent 通过工具调用完成“搜索表情包” “发送图片” “查询历史” “更新群档案”等动作，而不是把所有 能力写进一个巨大的提示词。这种“模型 + 工具”的结构与 ReAct 思路一致，便于扩展与审计。

4.3 群聊参与决策

决策链由四层组成：

1. **抽样层：**每条普通文本消息按群级概率独立抽样；@、回复 Bot、连续对话窗口等显式信号可绕过抽样。
2. **Gatekeeper：**使用 Qwen-Flash 判断“是否适合接话” “是否承接 Bot” “发言对象是谁”，输出 JSON。
3. **参与策略：**同一群两次判断至少间隔 20 秒；发言后有 45–120 秒冷却；10 分钟内主动消息不超过 3 条；Bot 发言占比过高时退让。
4. **群相处感受：**根据长期互动调整主动意愿，但受基础概率与上限约束，避免越聊越刷屏。

> **设计要点：** “高概率”只意味着更频繁地进入判断，不意味着必然发言。真正的发言仍由 Gatekeeper 与 冷却策略控制，从而兼顾“活跃”与“不打扰”。

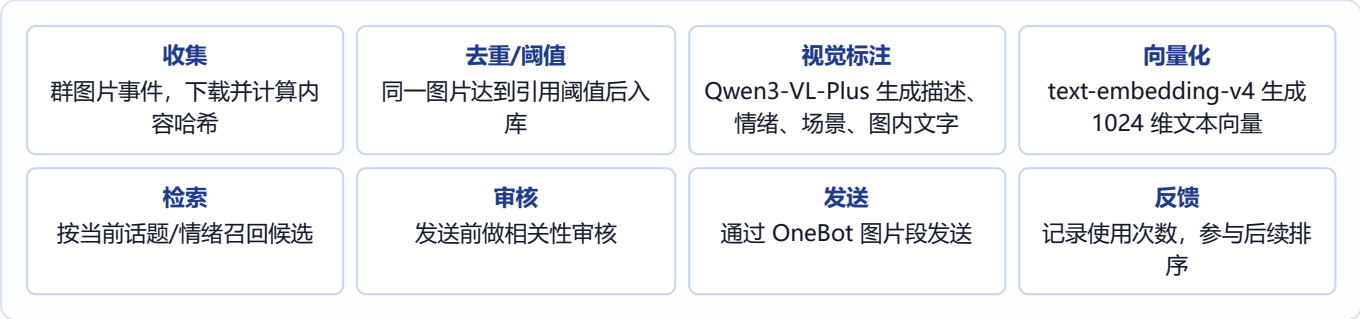
4.4 上下文压缩与长期记忆

系统将上下文分为五层，按需注入，避免把全部历史塞进模型：

层级	内容	作用
L1 最近窗口	最近约 20 条群消息	保证即时语境与指代解析
L2 群体档案	长期话题、成员特征、内部梗、氛围	让 Bot 理解群文化
L3 明确记忆	群友要求记住的称呼、偏好、约定	跨会话保持一致性
L4 RAG	按语义检索的历史聊天片段	回答“之前聊过什么”类问题
L5 摘要/预算	群档案摘要、长上下文阈值与 token 预算	控制成本与延迟

L2/L4/L5 由插件在后台异步维护：消息先入库，向量化与摘要不阻塞当前回复；当上下文超过阈值时， 使用摘要与检索结果替代早期原文。这样既保留长期记忆，又避免上下文无限增长。

4.5 表情包学习流水线



项目采用 `MEME_EMBEDDING_MODE=text`，即用“描述文本”做向量检索，而不是直接做图找图。 这样只需要文本 Embedding，部署更轻；视觉理解由 Qwen3-VL-Plus 在标注阶段完成。表情包入库阈值设为 2 次， 在“尽快学习常用表情”和“避免收集一次性照片”之间取折中。

4.6 模型角色编排

角色	模型	用途	设计考虑
主聊天 / Agent	qwen3.7-plus	对话、工具调用、复杂语境	支持长上下文、视觉输入与 function calling
Gatekeeper	qwen-flash	是否接话、对象判断	高频、低成本、低延迟
摘要	qwen-flash	群档案与长期记忆压缩	后台异步，成本可控
视觉标注	qwen3-vl-plus	表情包/图片理解	多模态输入，输出结构化标签
Embedding	text-embedding-v4	聊天与表情包描述向量化	1024 维，与 Qdrant 集合元数据一致

4.7 安全设计

- API Key 仅存在于 `.env` 与运行环境， `.gitignore` 排除；仓库扫描不含真实密钥。
- OneBot 3001、Qdrant 6333 仅在 Docker 网络内；NapCat WebUI 绑定 `127.0.0.1`。
- 文件/代码执行工具默认关闭，降低群友诱导读取本机文件或执行代码的风险。
- 群日限额、冷却与频率上限降低异常刷屏与费用失控风险。

5. 面向群聊体验的增量优化

上游插件已经提供了完整的群聊智能体能力。本项目只做少量、可复现、可回滚的增量优化，避免偏离成熟方案。所有优化均记录在 `patches/ai-groupmate-active-groups.patch` 与 `docs/OPTIMIZATIONS.md`。

5.1 单群活跃度补丁

上游的单群概率覆盖必须绑定“群级独立模型 API”，且本地抽样在 `ParticipationPolicy.candidate()` 中硬编码 `min(0.1, ...)`。这导致重点群无法真正提高活跃度。项目增加 `active_group_probabilities` 配置，并在解析概率与抽样上限两处做最小改动：

```
# config.py
+ active_group_probabilities: dict[str, float] = Field(
+     default_factory=dict,
+     description="按群号覆盖主动接话抽样概率",
+ )

# group_model_config.py
+ boosted = global_config.active_group_probabilities.get(str(group_id))
+ if boosted is not None:
+     return max(0.0, min(1.0, float(boosted)))

# participation.py（允许重点群突破 0.1 上限）
- probability = min(0.1, probability * affinity_multiplier(key))
+ probability = min(max(0.1, probability), probability * affinity_multiplier(key))
```

这样普通群仍被限制在 0.1，重点群可以配置到 0.5 或 1.0；但 Gatekeeper、20 秒判断间隔、45–120 秒发言冷却、 10 分钟 3 条上限仍然生效。补丁以 unified diff 形式提供，避免直接 vendoring 上游源码。

5.2 身份与昵称对齐

上游使用 `AI_GROUPMATE__BOT_NAME` 做直接称呼识别与“停止插话”判断。项目初期该值为占位符 “小群友”，而群友实际称呼 Bot 为“yuki”，导致“yuki 在吗”没有被识别为 direct address。将 `BOT_NAME` 与 `NICKNAME` 对齐后，显式称呼会绕过随机抽样直接进入 Gatekeeper， 显著改善“叫了不理”的体验。

5.3 人设与表达风格

人设提示词从通用“群友”细化为：短句、一次一个点、懂 ACG/动画/游戏梗、不懂先看上下文、不硬编、 不客服腔、不每句总结、不被叫到时自然回应。该优化只修改 `AI_GROUPMATE__PERSONALITY_SETTING`，不改变代码。

5.4 上下文与表情包参数

参数	调整	目的
`GROUP_MESSAGE_DEBOUNCE_SECONDS`	1.5 秒	同一话题连续消息合并，减少碎片化判断
`CONTINUOUS_CONVERSATION_MINUTES`	8 分钟	延长连续对话窗口，追问更容易被接住
`MEDIA_VECTORIZE_MIN_REFERENCES`	2 次	更快学习常用表情包，同时避免收集所有图片
`REPLY_PROBABILITY`	0.05	降低非重点群打扰
`ACTIVE_GROUP_PROBABILITIES`	重点群 1.0	让目标群每条消息都进入判断
`CHAT_TEMPERATURE`	0.75	略增表达变化，减少模板化

5.5 优化前后对比

维度	优化前	优化后
单群最高抽样概率	0.1（硬上限）	重点群可配置到 1.0；普通群仍 0.1
昵称识别	占位名，叫 yuki 不命中	BOT_NAME/NICKNAME 对齐，显式称呼直达判断
连续对话	5 分钟	8 分钟
表情包收录	3 次引用	2 次引用
非重点群干扰	全局 0.10	全局 0.05

6. 部署与运维

6.1 本地开发环境

项目	配置
操作系统	Windows 10/11
Python	3.12（虚拟环境 `bot/.venv`）
QQ / NapCat	OneKey 便携版 QQ 9.9.33 + NapCat 4.18.41
NoneBot	2.5.0 + nonebot-adapter-onebot 2.4.6
插件	nonebot-plugin-ai-groupmate 2.5.16（editable + 补丁）
向量库	Qdrant 1.20.0（本地二进制）

6.2 阿里云 ECS 部署

项目	配置
操作系统	Ubuntu 24.04.2 LTS
资源	1.6 GiB RAM；2 GiB swap；约 59 GiB 磁盘
容器运行时	Docker 29.9.1；Docker Compose v5.6.0
服务编排	qdrant / napcat / bot 三个容器，独立 `qqnet` 网络
数据持久化	`deploy/data/{qdrant,napcat,bot,bot-config,bot-cache}`
自动迁移	`deploy/entrypoint.sh` 启动前执行 `nb orm upgrade`

6.3 端口规划

端口	服务	暴露策略	状态
18080	NoneBot / 插件 WebUI	公网（安全组入方向）	**待放行**
16099	NapCat WebUI	仅 127.0.0.1，SSH 隧道	**正常**
3001	OneBot WebSocket	Docker 内部	**正常**
6333	Qdrant HTTP/gRPC	Docker 内部	**正常**

> **当前唯一部署待办**：阿里云安全组尚未放行 TCP 18080，因此公网访问 `http://<ECS_PUBLIC_IP>:18080/ai-groupmate/usage` 会超时；服务器本地访问返回 200。 需要在 ECS 控制台添加入方向规则：TCP 18080，源为管理端公网 IP（推荐）或 0.0.0.0/0。

6.4 启动与登录流程

1. 服务器克隆仓库，复制 `bot/.env.example` 为 `bot/.env`，填写百炼 Key、群号、token。
2. 执行 `bash deploy/init.sh`：生成 NapCat token、渲染 `onebot11.json`、构建/启动容器。
3. NapCat 首次登录无 GUI，通过容器日志或 SSH 隧道打开 WebUI 获取二维码。

4. 用 Bot 小号扫码；NapCat 写入 `onebot11_<QQ>.json`，3001 开始监听。
5. Bot 连接 `ws://napcat:3001`，日志出现 `Bot <QQ> connected`。

6.5 运维与故障恢复

场景	处理
容器重启	`restart: unless-stopped`；Bot 启动前自动 ORM 迁移，迁移幂等
QQ 掉线	NapCat 保留配置，重新扫码/快速登录；Bot 自动重连 3001
数据库未迁移	entrypoint 已内置 `nb orm upgrade`，避免启动失败循环
公网不可达	先查安全组，再查容器端口映射与本地 200 响应
内存紧张	2 GiB swap 兜底；监控 `docker stats`，必要时拆分 Qdrant 或升级内存

7. 测试与结果

7.1 测试环境

项目	值
测试日期	2026-10-28
服务端	阿里云 ECS，Ubuntu 24.04.2 LTS，Docker 29.9.1 / Compose v5.6.0
客户端	Windows 10/11，Python 3.12，OneKey QQ 9.9.33 + NapCat 4.18.41
模型服务	阿里云百炼 OpenAI 兼容接口
测试群	目标群（群号脱敏）

7.2 功能测试

编号	用例	预期	结果
T1	百炼 CLI 登录与 Key 同步	`authenticated=true`，四个角色共用 Key	**通过**
T2	Qdrant 启动与集合创建	`chat_collection_v2`、`media_collection_text` 存在	**通过**
T3	NapCat 扫码登录	3001 监听，生成 `onebot11_<QQ>.json`	**通过**
T4	Bot 连接 OneBot	日志出现 `Bot <QQ> connected`	**通过**
T5	群消息接收	文本、图片、回复、@ 均能进入 Matcher	**通过**
T6	自主接话与 LLM 回复	目标群出现多条自然回复	**通过**
T7	表情包发送	NapCat 日志出现图片发送记录	**通过**
T8	单群活跃度补丁	目标群 Gatekeeper 调用显著增加，其他群仍受 0.1 限制	**通过**
T9	数据库自动迁移	空数据卷首次启动不崩溃	**通过**
T10	WebUI 本地访问	服务器本地 18080 返回 200	**通过**
T11	WebUI 公网访问	公网 18080 返回 200	**待安全组**

7.3 实际运行证据

服务器部署后，Bot 在目标群连续参与对话。以下为脱敏后的回复片段，用于说明链路与语气：

语境追问

“不太确定你指的是哪个‘奶娃’，这个词可能有几种意思……”

短句接话

“咋了？”

自然互动

“被当面 debug 了，有点尴尬。”

同时观察到 NapCat 日志中的图片发送记录，说明表情包链路可用；Bot 日志中的 Gatekeeper 记录显示目标群持续进入“是否接话”判断，而非仅靠 @ 触发。

7.4 资源占用

容器	CPU	内存	状态
qq-ai-bot-bot-1	1.12%	327.8 MiB	**running，0 重启**
qq-ai-bot-napcat-1	0.46%	248.0 MiB	**running，0 重启**
qq-ai-bot-qdrant-1	0.02%	19.28 MiB	**running，0 重启**
宿主机	—	已用 1.2 GiB / 可用 337 MiB；swap 0 B	**稳定**

7.5 问题与处理

问题	原因	处理
NapCat 注入后无窗口	系统 QQ 9.9.26 与 NapCat 包不匹配	改用 OneKey 便携版 QQ 9.9.33 + NapCat 4.18.41
Bot 启动即退出	空数据卷未执行 ORM 迁移	容器 entrypoint 启动前自动 `nb orm upgrade`
公网 18080 超时	阿里云安全组未放行	待添加 TCP 18080 入方向规则
NapCat `contentHead` 警告	个别消息结构解析异常	非致命；后续消息仍正常收发，持续观察

7.6 测试结论

系统在本地与服务器均完成了从 QQ 登录、消息接入、自主判断、模型回复、表情包发送到 WebUI 访问的闭环。在 1.6 GiB 内存的 ECS 上，三个容器稳定运行且未触发 swap；目标群的活跃度优化达到预期。 当前唯一阻塞项是公网 18080 的安全组规则，不影响服务器本地与群内功能。

8. 安全、合规与风险

8.1 账号与平台风险

NapCat 属于第三方 QQ 协议端，不属于腾讯官方支持范围。使用自动化账号可能触发风控、限制登录或封禁。项目明确建议使用小号，不登录主账号，不进行高频骚扰、批量加群、广告等行为，并遵守 QQ 用户协议与群规。

8.2 密钥与数据安全

对象	措施
百炼 API Key	仅存于 `.env` / 环境变量；仓库 `.gitignore` 排除；泄漏后立即轮换
OneBot token	随机生成；NapCat 与 Bot 一致；不提交仓库
QQ 登录态	保存在服务器数据卷；限制目录权限；不导出、不分享
聊天/表情包数据	本地 SQLite/Qdrant；告知群成员；按群规处理；可定期清理
WebUI	18080 仅对管理端 IP 开放；插件 WebUI token 保护；NapCat WebUI 仅本机

8.3 提示词注入与工具滥用

群聊内容不可信，可能包含“忽略之前的指令”“读取本机文件”等提示词注入。项目默认关闭文件/代码执行工具， 仅保留聊天、检索、表情包与记忆等必要能力；模型输出经过工具白名单约束，避免直接执行任意系统命令。

8.4 成本与滥用控制

- 全局低概率抽样 + 重点群高概率，避免全群高频调用。
- 每群每日模型回复上限、Gatekeeper 判断间隔、发言冷却与 10 分钟条数上限。
- 视觉标注与向量化在后台异步执行，避免阻塞与重复调用。
- 使用 Qwen-Flash 承担高频判断，主模型只处理真正需要回复的轮次。

8.5 风险矩阵

风险	概率	影响	缓解
QQ 账号被风控	中	高	小号、低频、遵守群规、避免敏感操作
API Key 泄漏	低	高	不入库、最小权限、轮换、日志脱敏
群内提示词注入	中	中	关闭危险工具、工具白名单、输出审核
低配服务器 OOM	中	中	swap、容器限额、监控、拆分 Qdrant
NapCat 版本不兼容	中	高	OneKey 便携版、固定版本、上线前验证

9. 局限性与后续工作

9.1 当前局限

- **资源紧张**：1.6 GiB 内存对 QQ + Bot + Qdrant 偏紧，虽然当前稳定，但高峰或重建索引时余量有限。
- **评估主观**：目前以实际群聊观察为主，缺少标注集与自动指标，难以量化“接话质量”。
- **Gatekeeper 保守**：为减少打扰，部分本可参与的闲聊会被拒绝；概率与提示词仍需按群调优。
- **单实例**：当前单 Bot、单群重点优化，尚未支持多 Bot/多租户隔离。
- **公网入口未完成**：18080 仍待安全组放行；NapCat WebUI 依赖 SSH 隧道。

9.2 后续工作

1. **可观测性**：接入 Prometheus/Grafana 或轻量日志面板，记录 Gatekeeper 命中率、回复率、延迟、token 成本。
2. **评估体系**：构建小规模群聊标注集，评估“该不该接”“接得是否自然”“是否重复/冒犯”，做 A/B 对比。
3. **记忆优化**：评估多模态 Embedding、Rerank 与分层摘要的收益；对表情包做图找图与视觉相似检索。
4. **资源优化**：将 Qdrant 拆到独立实例或使用托管向量库；为容器设置内存限额与告警。
5. **部署自动化**：补齐安全组 API 自动化、CI 构建、镜像版本固定与回滚脚本。
6. **多实例**：按群隔离配置、数据与模型配额，支持多个 Bot 账号。

10. 结论

本项目以 NapCat + OneBot v11 + NoneBot2 + ai-groupmate 为核心，构建了一个能够在 QQ 群聊中自主观察、 判断与参与的智能体。系统通过“抽样—Gatekeeper—Agent”的分层决策控制成本与打扰，通过最近窗口、群体档案、 明确记忆与 RAG 实现长上下文压缩，通过视觉标注与文本向量实现表情包学习，并通过 Docker Compose 完成 阿里云 ECS 的容器化部署。

针对真实使用中的问题，项目做了单群活跃度补丁、身份对齐、人设与参数调优等小步优化，并在服务器上完成了消息收发、群聊回复、表情包发送、WebUI 与自动迁移的验证。当前系统在 1.6 GiB 内存的 ECS 上稳定运行，唯一待办是安全组放行 18080。后续工作将围绕评估体系、可观测性、资源优化与多实例能力展开。

11. 参考文献

1. NoneBot. NoneBot2 文档. https://nonebot.dev/
2. OneBot. OneBot v11 标准. https://github.com/botuniverse/onebot-11
3. NapNeko. NapCatQQ. https://github.com/NapNeko/NapCatQQ
4. yaowan233. nonebot-plugin-ai-groupmate. https://github.com/yaowan233/nonebot-plugin-ai-groupmate
5. Qdrant. Qdrant Documentation. https://qdrant.tech/documentation/
6. 阿里云. 百炼 / Model Studio 文档. https://help.aliyun.com/zh/model-studio/
7. 阿里云. ECS 安全组文档. https://help.aliyun.com/zh/ecs/user-guide/security-groups
8. Docker. Compose file reference. https://docs.docker.com/compose/compose-file/
9. Lewis P., Perez E., Piktus A., et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 2020.
10. Yao S., Zhao J., Yu D., et al. ReAct: Synergizing Reasoning and Acting in Language Models. ICLR 2023.
11. LangChain. LangChain Documentation. https://python.langchain.com/

12. 附录

附录 A：仓库结构

```
qq-ai-groupmate/
├─ bot/                  NoneBot 项目、配置模板
├─ config/               NapCat onebot11 模板
├─ deploy/               Dockerfile、Compose、entrypoint、服务器说明
├─ patches/              ai-groupmate 单群活跃度补丁
├─ scripts/              安装、启动、自检、打补丁
├─ tools/                Key 同步、群/配置检查
├─ docs/                 优化说明
├─ LICENSE / NOTICE.md / SECURITY.md
└─ README.md
```

附录 B：关键配置（脱敏）

```
AI_GROUPMATE__BOT_NAME=<BOT_NAME>
AI_GROUPMATE__REPLY_PROBABILITY=0.05
AI_GROUPMATE__ACTIVE_GROUP_PROBABILITIES={"<TARGET_GROUP_ID>": 1.0}
AI_GROUPMATE__GROUP_MESSAGE_DEBOUNCE_SECONDS=1.5
AI_GROUPMATE__CONTINUOUS_CONVERSATION_MINUTES=8
AI_GROUPMATE__MEDIA_VECTORIZE_MIN_REFERENCES=2
AI_GROUPMATE__CHAT_MODEL=qwen3.7-plus
AI_GROUPMATE__FLASH_MODEL=qwen-flash
AI_GROUPMATE__TAGGING_MODEL=qwen3-vl-plus
AI_GROUPMATE__EMBEDDING_MODEL=text-embedding-v4
AI_GROUPMATE__EMBEDDING_DIMENSION=1024
AI_GROUPMATE__MEME_EMBEDDING_MODE=text
```

附录 C：Docker 端口与启动

```
# deploy/docker-compose.yml（节选）
services:
  bot:
    ports: ["18080:8080"]
    volumes:
      - ./data/bot:/app/data
    env_file: ["../bot/.env"]
  napcat:
    ports:
      - "127.0.0.1:16099:6099"
      - "127.0.0.1:3001:3001"
  qdrant:
    image: qdrant/qdrant:v1.20.0

# 首次部署
bash deploy/init.sh
# 查看日志
sudo docker compose --env-file deploy/deploy.env -f deploy/docker-compose.yml logs -f bot napcat
```

附录 D：补丁摘要

```
patches/ai-groupmate-active-groups.patch
  src/nonebot_plugin_ai_groupmate/config.py            +4
  src/nonebot_plugin_ai_groupmate/group_model_config.py +3
  src/nonebot_plugin_ai_groupmate/participation.py      +1/-1
```

附录 E：缩略语

缩写	全称 / 含义
WS	WebSocket
LLM	Large Language Model，大语言模型
VLM	Vision-Language Model，视觉语言模型
RAG	Retrieval-Augmented Generation，检索增强生成
ORM	Object-Relational Mapping，对象关系映射
ECS	Elastic Compute Service，阿里云弹性计算服务
QPS/TPM	Queries Per Second / Tokens Per Minute

报告结束。代码与部署文件：https://github.com/miaom4702-art/qq-ai-groupmate